

The `featuretree` IR: an Open Feature-Tree Format, Emitted Natively into Several CAD Systems and Checked by Re-Reading

Daniel Newcome

punkfab

<https://github.com/punkfab/featuretree>

October 2026 (version 2)

Abstract

When a program or a language-model agent writes a part, a person still has to be able to open it, see how it was built, and change one dimension without starting again. That needs a shared, small description of the feature tree — the “timeline” every history-based CAD system shows — that an agent can emit, each CAD system can rebuild natively, and an edit made in any of them can be read back into. Feature-based modelling has been studied since the 1980s and no complete vocabulary exists; we publish a deliberately small one and a rule for growing it. The `featuretree` IR is a declarative JSON list of named features — sketches with arcs, pad, pocket, fillet, revolve and placed cuts, with draft — whose references to faces and edges are stored as queries resolved against live geometry, never as kernel identifiers. One IR is emitted as a native, editable tree in FreeCAD, build123d and Onshape, and as scripts for Autodesk Fusion and SolidWorks. Every emitted part is checked the same way: STEP export, a volume and Boolean-free IoU gate against a reference build, and a per-feature volume trace that names the first feature where two backends disagree. We report the measurements, including the failures. On the eleven trees recovered from the NIST MBE PMI corpus FreeCAD reproduces the reference on 10 of 11; Onshape, limited by its free plan’s API quota, has run 3 and passes 1. An edit made in the FreeCAD tree, read back by feature name and rebuilt by the reference, agrees on 43 of 46 single-dimension edits; before a defect that this measurement found was fixed, it agreed on 27. Of the 240 features in the designers’ own trees of the same parts, 52% have a kind of their own in the IR; three more kinds (hole, pattern, chamfer) would bring that to 94%. Running the backends against each other found six defects, including one in the reference backend and two rules the specification had never stated. Neutral feature exchange is not new — it goes back to the macro-parametric work of 2003, and a closely related system appeared this year — so we claim no novelty. We offer an open specification, an MIT-licensed implementation, and measurements that can be reproduced.

1 Introduction

Moving a parametric part between CAD systems loses its design history. STEP, the neutral exchange standard, has carried shape reliably for decades; its parts for construction history (ISO 10303-55/108/111) were specified but, as their own authors reported, not adopted by commercial translators [29]. The alternatives that did ship — neutral modelling commands

replayed through each system’s API [26, 34], and commercial bridges such as Proficiency [31] — showed that the approach works, and identified the hard problems: feature mapping, persistent naming and constraint translation [34].

The question has come back for a new reason. Parts are now written by programs and by language-model agents, and most of that output is either code for one kernel (CadQuery, FreeCAD Python) or bare geometry [1, 33, 44]. A person reviewing such a part needs to nudge it: open it in the system they know, find the feature, change a dimension, and hand the result back. Bare geometry does not allow that, and code for one kernel allows it only to someone who reads that code. What is needed is a representation of the feature tree itself that is small enough for a program to emit, explicit enough for several CAD systems to rebuild natively, checkable, and able to take an edit back from any of them. Which features such a representation should contain is an old question: feature-based modelling was surveyed thirty years ago [35, 39], the vendors’ vocabularies still differ, and no list is complete. We do not try to settle it. We fix a small subset, measure how far it reaches, and state how it grows (§7). CADIR, which appeared in August 2026, targets the same need with an executable Python IR and adapters for FreeCAD, SolidWorks and Fusion [20]. `featuretree` was developed independently (public since June 2026) and makes different choices; we describe them and measure the result.

Contributions.

1. **A specification** (§2): a declarative, non-executable JSON feature list with seven feature kinds, query-based references, fixed coordinate and sign conventions, and well-formedness rules with a validator.
2. **Five backends** (§3): build123d (the reference), FreeCAD and Onshape, which we run and measure, and generated scripts for Fusion and SolidWorks, which we have not yet run.
3. **One verification method for every backend** (§5): STEP export, a volume and Boolean-free IoU gate, a per-feature volume trace, and read-back of native files through two independent decoders.
4. **A mapping to each system’s timeline** (§4): what every IR kind is called in each tree, which dimensions read back by name, and a measured edit round trip.
5. **Measurements** (§6): the backends on the eleven recovered NIST trees and on coverage parts; the six defects that cross-backend disagreement exposed; the edit round trip; a two-decoder study of NIST’s native SolidWorks files; and how much of the designers’ own feature vocabulary the IR covers.
6. **A rule for growing the vocabulary** (§7).

2 The IR

A part is `{"name": ..., "features": [...]}`: an ordered list of named features. Order is construction order; a feature may refer only to earlier features, and only by name. Table 1 lists the kinds. Listing 1 is a complete part.

Kind	Parameters and meaning
sketch	circles (c_x, c_y, r), rects (w, h, c_x, c_y), polys (closed loops of $(x, y[, b])$, b = bulge); on plane XY or XZ, or on a face given by the query {"face_of", "side": top bottom}
pad	extrude a sketch: length , symmetric , taper (deg)
pocket	cut a sketch: through or blind length , taper
fillet	radius on the edges selected by a query, e.g. the largest circle on the top
revolve	revolve an XZ profile about Z by angle
polar_pocket	count cylinders tangent to a circle (roller pockets)
prism_cut	cut a profile drawn on the plane (origin , normal , xdir), depth along the normal, taper

Table 1: Feature kinds. Lengths in millimetres, angles in degrees.

Listing 1: A complete part: a plate, a pocket in its top face, and a fillet.

```
{
  "name": "plate", "features": [
    {
      "kind": "sketch", "name": "outline", "plane": "XY",
      "polys": [[[-20,-15],[20,-15,0.4142],[20,15],[-20,15]]],
      "kind": "pad", "name": "body", "sketch": "outline", "length": 10,
      "kind": "sketch", "name": "recess_sk", "on": {"face_of": "body", "side": "top"},
      "circles": [[0, 0, 6]],
      "kind": "pocket", "name": "recess", "sketch": "recess_sk", "through": false, "length": 4,
      "kind": "fillet", "name": "round", "radius": 1, "select": {"circles": "top_outer"}}
  ]
}
```

References are queries. A face-attached sketch names “the top face” and a fillet names “the largest circular edge on top”, and every backend resolves the query against its own live geometry each time the part is built. This is the approach of generic naming [6] and of geometry-based semantic identifiers [42], and of the selectors in CadQuery, build123d and Onshape’s FeatureScript; it avoids tracking kernel names through history [17, 24]. The known cost applies: after a large edit a query can resolve to a different entity.

Conventions. Coordinates of a face-attached sketch stay global: a circle at $(0, 0)$ on the top face is above the origin, whatever frame the backend gives that face. XZ maps (u, v) to $(u, 0, v)$. A pad grows $+Z$, or $-Z$ from a bottom face. A bulge is the DXF arc factor $\tan(\theta/4)$, its arc’s midpoint on the chord’s left normal, and $|b| < 10^{-4}$ is a straight line. A positive taper shrinks the profile along the extrusion (for a pocket, as it goes deeper). A sketch’s region is its loops under the nesting rule: a loop inside another is a hole, an island in a hole is material again, and separate loops are separate regions.

Well-formedness. `ir.validate(spec)` checks what every backend is entitled to assume: unique names; references to earlier sketches; closed loops (at least three segments, or two if one is an arc) with no zero-length segment and no self-intersection; within one sketch, loop boundaries that never touch or cross; a taper only on a blind pocket; an orthogonal placement frame. Two of these rules (non-touching loops and the bulge threshold) were added after backends disagreed about parts that broke them (§6.2).

Backend	Driven by	References become	Status
build123d	in-process Python (OCCT)	queries on the live solid	reference
FreeCAD 1.0	headless <code>freecadcmd</code> , PartDesign	<code>FaceN/EdgeN</code> re-resolved per build	measured
Onshape	REST v6 feature JSON	FeatureScript query strings	measured
Fusion	generated script (<code>adsk.fusion</code>)	queries evaluated in the script	not yet run
SolidWorks	generated script (COM, pywin32)	queries evaluated in the script	not yet run

Table 2: Backends. “Measured” means built and gated in this paper.

3 Backends

Table 2 summarises them. **build123d** is the reference: every other backend is compared with its build of the same IR. **FreeCAD** writes a native PartDesign body in which each feature’s label is its IR name, so a dimension edited in the GUI reads back by name. **Onshape** receives native Sketch, Extrude, Fillet, Revolve and Plane features. Every reference is a FeatureScript query string rather than an entity identifier: a profile is `qContainsPoint(qSketchRegion(id), p)` for an interior point p of each material region, a face-attached sketch plane is the planar Z -normal face farthest along $\pm Z$, and a fillet’s edges are the circular edges containing points computed from the same rule the reference uses. Onshape re-evaluates them on every regeneration, so the tree stays editable. Placed cuts on oblique planes and polar pockets are not yet emitted. **Fusion** and **SolidWorks** cannot be driven from outside the program, so the emitter writes a self-contained script: the IR as data, the reference volume after every feature, and a small runtime that re-authors each feature and resolves each query inside the program, then exports STEP and a report. The shared geometry those runtimes draw from is tested by replaying it with build123d, but the scripts have not yet run in either program, so we report no results for them.

We looked for a way to write a SolidWorks file without SolidWorks. The open-source decoder we use to read `.sldprt` files, `cadmjpeg`, also lists `.sldprt` and `.f3d` as write targets. At version 0.6.0 its writers decline a plain box read from STEP (“SLDPRT semantic writer does not encode explicit edge parameter ranges”; for Fusion, a file with no native source cannot be serialised), the same refusal is in its source on 4 October 2026, and none of its 24 public forks changes the writer. So today the open route reads SolidWorks files but does not write them, and a SolidWorks result still needs the program.

4 The IR as a shared timeline

Each history-based CAD system shows the user an ordered list of features: the model tree in FreeCAD, the feature list in Onshape, the timeline in Fusion, the FeatureManager tree in SolidWorks. The IR is that list with the vendor removed. Table 3 gives the correspondence the backends implement. Every emitted feature carries its IR name as its name in the target’s tree, and that name is the only key used in either direction.

The edit loop. A person changes a dimension in the target’s own interface. A reader for that target returns `{feature name: {parameter: value}}`; `ir.update_from_params` writes those values into the IR; every other backend rebuilds from the IR. Nothing else flows back: a feature added, deleted or reordered in the target is not read, and sketch geometry other than circle radii

IR kind	FreeCAD	Onshape	Fusion, SolidWorks	Reads back
sketch	Sketch	Sketch	Sketch	circle radii
pad	Pad	Extrude (new/add)	Extrude (join), Boss- Extrude	length
pocket	Pocket	Extrude (remove)	Extrude (cut), Cut- Extrude	length
fillet	Fillet	Fillet	Fillet	radius
revolve	Revolution	Revolve	Revolve	angle
prism_cut	plane, sketch, Pocket	plane, sketch, Extrude (remove)	plane, sketch, cut	depth
polar_pocket	n subtractive cylinders	not emitted	n placed cuts	–

Table 3: What each IR kind becomes in each system’s tree, and the driving dimension that is read back by feature name. The Fusion and SolidWorks columns describe the generated scripts, which have not run.

is not read. Readers exist for FreeCAD and Onshape, and inside the Fusion and SolidWorks scripts. §6.4 measures the FreeCAD loop.

What the table hides. The mapping is one-to-one only for the first five rows. A placed cut becomes three tree entries in every target, and a polar pocket becomes n ; a person reading the tree sees the parts, not the IR feature. The reverse problem is larger: a designer’s hole or pattern feature has no kind of its own and arrives as plain cuts (§6.5).

5 Verification

A backend’s output is exported to STEP and compared with the reference build of the same IR. It passes (VERIFIED) when volumes agree within 0.5 % and the intersection-over-union is at least 99.5 %. IoU is estimated by classifying uniformly sampled points against both solids [41], never by a Boolean, because OpenCASCADE’s Booleans returned wrong unions on these parts; robustness failures of this kind are well documented [12, 47]. The volume check is the idea behind CAX-IF’s geometric validation properties [4] and Sap and Shapiro’s invariant-property testing [36]; IoU additionally catches misplaced material that leaves volume unchanged.

Each backend also records the volume after every feature. When a part fails, the first feature whose volume leaves the reference is where to look; every defect in §6.2 was found that way.

For native files (.sldprt, .f3d) a third check applies: the file is decoded to STEP by two independent decoders, and its stored feature history is matched to the IR by name, kind and driving value. Comparing two implementations of the same function is differential testing [25, 46]; it answers the oracle problem [2] that a single decoder’s own consistency check cannot (§6.3).

6 Results

6.1 FreeCAD and Onshape against the reference

The corpus is the eleven feature trees that `featuretree`’s recogniser recovers from the NIST MBE PMI test parts [19], 2 to 144 features each (the recogniser and its accuracy are the subject

Part	Features	Valid	FreeCAD			Onshape		
			Verdict	IoU %	$ \Delta V $ %	Verdict	IoU %	$ \Delta V $ %
CTC-01	54	yes	VERIFIED	100.00	0.00	53/54 built	99.36	0.58
CTC-02	144	yes	VERIFIED	99.90	0.05	93/144 built	0.41	98.26
CTC-03	64	no	VERIFIED	100.00	0.00	not run		
CTC-04	108	yes	VERIFIED	100.00	0.00	not run		
CTC-05	57	yes	VERIFIED	99.78	0.23	not run		
FTC-06	27	no	build fails			not run		
FTC-07	29	yes	VERIFIED	100.00	0.00	not run		
FTC-08	37	yes	VERIFIED	100.00	0.00	not run		
FTC-09	80	yes	VERIFIED	100.00	0.00	not run		
FTC-10	38	yes	VERIFIED	100.00	0.00	not run		
FTC-11	2	yes	VERIFIED	100.00	0.00	VERIFIED	100.00	0.00

Table 4: Each recovered NIST feature tree built by FreeCAD and Onshape and gated against the build123d reference build of the same IR (VERIFIED: $\text{IoU} \geq 99.5\%$ and $|\Delta V| \leq 0.5\%$). “Valid”: passes `ir.validate`. Onshape’s run is limited by the free plan’s daily API quota.

of a companion paper; here the trees are simply inputs). Table 4 gives FreeCAD’s result on every part and Onshape’s where it ran.

FreeCAD reproduces the reference on 10 of 11 parts. The exception is FTC-06 (build fails); §6.2 traces the cause.

Onshape has run on 3 parts and is VERIFIED on 1, FTC-11. On CTC-01 it builds 53 of 54 features; one placed cut, whose profile edge lies exactly on the part’s outer wall, is rejected by Onshape’s kernel, and the part reaches 99.36 % IoU, just under the gate. On CTC-02, the largest tree, it builds 93 of 144: ten of the stacked pads that make up the body regenerate with an error, most of the later cuts then have nothing to cut, and the result is far from the reference. We have not yet found why those pads fail. The free plan’s daily API quota (about 1,500 calls) is what limits this column: our emitter spends several calls on each feature, CTC-02 alone used a day’s quota, and the remaining eight parts have not run. Cutting the calls per feature is the next change to that backend.

Onshape was also run on coverage parts built to exercise every path it supports (Table 5). With the conventions pinned, every feature’s volume matches the reference to three decimals, with one exception: a pocket whose circle crosses a concave arc edge removes 4.175 mm^3 more in Onshape than the exact answer, computed independently, which build123d matches. We have not explained it.

6.2 What disagreement found

Running backends against each other is a test of each of them and of the specification. The trace located six defects, none of which any single backend reported:

1. **The reference was wrong.** build123d decided that a sketch was on the bottom face by its height being below zero, so a pad from a bottom face at $z = 0$, the common case, grew upward into the part. Onshape disagreed on the first such feature. The rule now uses the sketch’s declared side. No published result changed: all eleven recovered trees give identical reference volumes before and after.

Part	Feature	Onshape (mm ³)	Reference (mm ³)	Difference
plate (sample)	body	11,497.345	11,497.345	+0.000
	hole	11,497.345	11,497.345	+0.000
every kind	base	22,028.273	22,028.273	+0.000
	top_pk	21,784.056	21,788.231	-4.175
	hub	24,328.746	24,332.921	-4.175
	side_slot	24,157.241	24,161.417	-4.176
	thru	24,006.445	24,010.621	-4.176
	round	23,980.157	23,984.332	-4.175
taper A	base	12,000.000	12,000.000	+0.000
	up	12,432.802	12,432.802	+0.000
	down	12,668.025	12,668.025	+0.000
	sym	12,724.573	12,724.573	+0.000
taper B	base	12,000.000	12,000.000	+0.000
	pk_bottom	11,826.944	11,826.944	+0.000
	pk_top	11,632.467	11,632.467	+0.000
	side	11,550.769	11,550.769	+0.000

Table 5: Onshape coverage parts: volume after every solid feature against the build123d reference. The -4.175 offset enters at `top_pk` (a pocket crossing a concave arc edge) and is carried through; everything else agrees to three decimals.

2. **FreeCAD kept one solid.** A PartDesign body is one solid by default, so a sketch of several separate outlines padded only one of them, exactly halving the first feature of CTC-03 and CTC-04. FreeCAD 1.0’s `AllowCompound` restores the specified union; CTC-03 went from 2.5 % IoU to VERIFIED.
3. **A rule the specification lacked: loops must not touch.** Two recovered trees contain sketches whose loops share boundary. build123d unions loops one at a time and accepts them; FreeCAD rejects some (FTC-06). `ir.validate` now reports them; the recogniser should not produce them.
4. **A rule the specification lacked: near-zero bulges.** CTC-02 contains arcs of bulge 5×10^{-6} , radius about two kilometres. FreeCAD’s sketch solver fails on them. Below 10^{-4} a bulge now means a straight line in every backend; the reference volume of CTC-02 moves by $-0.000\,009\%$.
5. **Onshape’s draft direction** is the opposite of our first guess; a tapered boss grew where it should shrink. Pinned by matching the reference on tapered pads and pockets in both directions.
6. **Failure behaviour differs.** A cut that removes nothing is an error in Onshape and silently does nothing in build123d; the trace shows both.

6.3 Reading native files back: one decoder is not enough

NIST publishes the same test parts as native SolidWorks 2018 files. We decoded each to STEP with an open-source clean-room decoder, `cadmpeg` 0.6.0, and with Onshape’s commercial importer, and compared both with NIST’s own STEP of the part (Table 6). Onshape matches on 11 of 11 (IoU $\geq 99.93\%$); `cadmpeg` on 1. Four of `cadmpeg`’s wrong parts pass its own model

Part	Onshape import		cadmpeg 0.6.0		Solid operations	
	IoU %	ΔV %	IoU %	STEP	Designer	Recovered
CTC-01	100.00	-0.000	21.39	invalid	10	39
CTC-02	99.94	+0.009	3.47	invalid	42	89
CTC-03	100.00	-0.164	11.14	invalid	15	33
CTC-04	100.00	-0.001	22.37	no solid	24	93
CTC-05	100.00	-0.000	6.99	no solid	17	40
FTC-06	100.00	+0.000	25.83	invalid	21	18
FTC-07	99.93	+0.009	0.00	no solid	28	21
FTC-08	100.00	+0.008	6.95	invalid	30	32
FTC-09	100.00	-0.000	80.72	invalid	33	40
FTC-10	100.00	+0.001	26.46	no solid	19	25
FTC-11	100.00	-0.000	100.00	valid	1	1

Table 6: NIST’s native SolidWorks 2018 files decoded to STEP by two decoders and compared with NIST’s own STEP of each part; and the number of solid operations in the designer’s tree (as decoded) against the tree `featuretree` recovers from the STEP.

check and convert without complaint; four more yield a shell that is not a solid. Its decoding of the feature history, on the other hand, looked right on the same files. The practical rule we adopted: a native file’s geometry is judged by the commercial importer when available, its history by the open decoder. We reported the geometry failures upstream with a reproduction.

6.4 An edit, round the loop

For each part FreeCAD could build quickly (the two samples and eight NIST trees; CTC-02 and CTC-04 take minutes per rebuild and were left out) we took up to six editable dimensions spread through the tree. Each was set to 1.1 times its value in the FreeCAD file, alone, as a person would in the GUI; FreeCAD recomputed; every parameter was read back into the IR; and the reference rebuilt from the IR (Table 8).

43 of 46 edits agree. The first run agreed on only 27 of 46: 16 edits to placed cuts never reached the IR, because FreeCAD holds a placed cut as a Pocket and reports its depth as `length`, while the IR reader looked for `depth`. The edit was silently dropped and the rebuilt part was the old part. The same run found that the function for editing a sketch circle called a FreeCAD method that does not exist in version 1.0. Both are fixed, and both had been in the code since the round trip was first written; no test had exercised them on a recovered tree.

The three that still fail are all edits to the length of a base pad. On FTC-07 (two edits) FreeCAD’s recomputed body is invalid, with a negative volume, while the reference rebuilds a valid solid. On CTC-01 neither system produces a solid. The recovered trees place later cuts at absolute coordinates rather than relative to the pad they cut, which is the likely reason, but we have not diagnosed these and report them as failures.

6.5 How much of a designer’s vocabulary

The native SolidWorks files carry the designers’ own feature trees, which cadmpeg decodes. Table 7 counts their solid-modifying features over the eleven parts and says what the IR has for each. 126 of 240 (52%) are extrudes, fillets and revolves, which have kinds of their own. Almost

Designer’s feature	Count	Share %	In the IR
extrude	85	35.4	own kind: pad, pocket, prism_cut
hole	63	26.2	shape only, as a pocket or placed cut
fillet	37	15.4	own kind: fillet
pattern	27	11.2	shape only, as its instances, one cut each
draft	11	4.6	shape only, as taper on the extrusion it drafts
chamfer	10	4.2	shape only, as a placed cut
revolve	4	1.7	own kind: revolve
shell	1	0.4	shape only, as a pocket
rib	1	0.4	shape only, as a pad
combine	1	0.4	–
all	240	100.0	

Table 7: The solid-modifying features in the designers’ own trees of the eleven NIST SolidWorks files (as decoded by cadmpeg), and what the IR has for each. “Shape only” means the IR can hold the geometry but the feature loses its identity in the tree.

all of the rest are holes, patterns and chamfers (100 features): the IR can hold their shape, as cuts, but not the fact that the designer made one hole feature with a counterbore or one pattern of twelve. Those three kinds would bring the share with a kind of its own to 94%. Two cautions apply. The count says a kind exists, not that every option of the designer’s feature is covered: the IR’s fillet, for example, can select only circular edges on the top face. And eleven parts from one test suite modelled in one CAD system are a small sample of how people model.

The same gap shows in tree length. The trees our recogniser recovers from the STEP files are 2 to 4 times longer than the designers’ on the CTC parts (CTC-01: 39 operations against 10), mostly because one hole or pattern feature stands for many of our cuts (Table 6).

7 Growing the vocabulary

The subset is meant to grow, and §6.2 shows what an under-specified kind costs: two backends that each look correct and disagree. A kind is added when all of the following hold.

1. **Its meaning is written without reference to any kernel:** the parameters, their units, every sign and direction convention, and what happens at the edges (a cut that removes nothing, a zero length).
2. **The validator checks what backends may assume** about it.
3. **The reference backend builds it**, and a second backend on a different kernel agrees with the reference at that feature, in the volume trace, on parts written to exercise each convention in both directions.
4. **It has a row in Table 3:** a native feature of the same meaning in each target, or an explicit statement that the target gets a decomposition.
5. **Its driving dimensions read back by name**, and the edit round trip of §6.4 passes for it.

Part	Editable	Edited	Agree	Volumes differ	Reference rebuild failed
CTC-01	53	6	5	0	1
CTC-03	33	6	6	0	0
CTC-05	40	6	6	0	0
FTC-07	17	6	4	2	0
FTC-08	32	6	6	0	0
FTC-09	11	6	6	0	0
FTC-10	25	6	6	0	0
FTC-11	0	0	0	0	0
sample plate	2	2	2	0	0
sample poly	2	2	2	0	0
all	215	46	43	2	1

Table 8: The edit round trip. One dimension at a time is set to 1.1 times its value in the FreeCAD tree, every parameter is read back into the IR by feature name, and the reference rebuilds from the IR. “Agree”: the edited FreeCAD solid and the rebuilt reference are within 0.5% in volume. “Editable”: dimensions FreeCAD can drive; up to six per part were edited.

By this rule the next kinds are the ones Table 7 ranks: hole (with its counterbore, countersink and thread callouts as parameters), linear and circular pattern of an earlier feature, and chamfer. Each has a native feature of the same name in all four targets. A draft applied to existing faces, shell and rib are rarer in this sample and wait. Sketch constraints are a separate question that this vocabulary does not address.

8 Related work

Neutral feature exchange. STEP’s construction-history parts (ISO 10303-55/108/111/112) [14, 16, 29]; the macro-parametric approach and its test rallies between commercial systems [15, 26, 34]; Proficiency’s universal product representation [31]; BYU’s multi-user neutral database [40]. Interoperability’s cost motivated all of it [5]. **Closest system:** CADIR [20] records a construction graph from an executable Python IR and rebuilds native histories in FreeCAD, SolidWorks and Fusion, matching topology by geometric signature; **featuretree** differs in being a declarative data format, in storing queries rather than matched selections, in targeting Onshape and build123d, and in sharing its IR with a STEP recogniser. **CAD as a language.** Code-CAD (OpenSCAD, CadQuery, build123d, FeatureScript, KCL) [21] and programming-language treatments [7, 27, 28, 32]; learned construction sequences, mostly sketch and extrude [13, 33, 38, 43–45]. **Persistent naming** [3, 6, 8, 17, 24, 42]. **Language models for CAD** emit CadQuery, FreeCAD Python or FeatureScript for one system each [1, 10, 22, 23, 30]. **Exchange quality:** validation properties [4], invariant-property and query-based interoperability [11, 36, 37], benchmarks [9], and NIST’s conformance testing [18, 19].

9 Limitations

- **Fusion and SolidWorks have not run.** Their scripts exist and their geometry is replay-tested, but API behaviour, sign conventions and profile selection are unverified until a run passes the gate.

- **Onshape’s corpus run is incomplete** (3 of 11 parts) because of the free plan’s API quota; one of the three fails badly for a reason we have not found; and the backend does not yet emit oblique placed cuts or polar pockets.
- **The edit loop carries dimensions only.** Features added, removed or reordered in a target are not read back; the round trip was measured for FreeCAD only, on up to six edits per part, one at a time, each a 10% change.
- **The vocabulary is small.** About half of the features designers used on these parts arrive without their identity (§6.5).
- **The reference is OpenCASCADE, and so is FreeCAD.** Their agreement is partly self-consistency. Onshape (Parasolid) is the independent kernel.
- **Single bodies, no sketch constraints, no assemblies** in this IR; sketches arrive in the target unconstrained.
- **Queries can re-resolve wrongly** after large edits; we have not measured how often.
- **The corpus is eleven parts** and the trees are machine-recovered, not hand-authored.

10 Availability

Code, specification (`ir.py`), validator and every backend are MIT-licensed at <https://github.com/punkfab/featuretree>; the numbers here come from the commit tagged `ir-paper-v2`. The measurements are regenerated by:

- `paper/figures/freecad_corpus.py` and `onshape_corpus.py` (Table 4);
- `paper/figures/native_nist.py` (Tables 6 and 7);
- `paper/figures/nudge_roundtrip.py` (Table 8);
- `paper/ir/tables.py`, which writes every table and inline number.

The software is archived as 10.5281/zenodo.22848721; the companion recovery paper is 10.5281/zenodo.22848792.

How this was built. The work was iterative: we steered the design with Claude (Anthropic) over many rounds, pushing on correctness, generality and performance; Claude wrote and debugged the code and ran the measurements.

References

- [1] Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. Generating CAD code with vision-language models for 3D designs. In *International Conference on Learning Representations (ICLR)*, 2025.
- [2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5): 507–525, 2015. doi: 10.1109/TSE.2014.2372785.

- [3] Rafael Bidarra and Willem F. Bronsvort. Persistent naming through persistent entities. In *Geometric Modeling and Processing (GMP 2002)*, pages 233–240, 2002. doi: 10.1109/GMAP.2002.1027515.
- [4] Jochen Boy, Phil Rosché, and Doug Cheney. Recommended practices for geometric and assembly validation properties, release 4.2. CAx Implementor Forum, October 2014. URL https://www.mbx-if.org/home/wp-content/uploads/2024/05/rec_prac_gvp_v42.pdf.
- [5] Smita B. Brunnermeier and Sheila A. Martin. Interoperability costs in the US automotive supply chain. *Supply Chain Management: An International Journal*, 7(2):71–82, 2002. doi: 10.1108/13598540210425821.
- [6] Vasilis Capoyleas, Xiangping Chen, and Christoph M. Hoffmann. Generic naming in generative, constraint-based design. *Computer-Aided Design*, 28(1):17–26, 1996. doi: 10.1016/0010-4485(95)00014-3.
- [7] Dan Cascaval, Rastislav Bodík, and Adriana Schulz. A lineage-based referencing DSL for computer-aided design. *Proceedings of the ACM on Programming Languages*, 7(PLDI): Article 168, 2023. doi: 10.1145/3591223.
- [8] Shahjadi Hisan Farjana and Soonhung Han. Mechanisms of persistent identification of topological entities in CAD systems: A review. *Alexandria Engineering Journal*, 57(4): 2837–2849, 2018. doi: 10.1016/j.aej.2018.01.007.
- [9] Salvatore Gerbino and Alessandro Brondi. Interoperability issues among CAD systems: A benchmarking study of 7 commercial MCAD software. In *Proceedings of DESIGN 2004, the 8th International Design Conference (DS 32)*, pages 617–626, 2004.
- [10] Ammon I. Hepworth and John M. Gauch. Iterative generation of feature-based CAD models using an LLM with domain-specific constraints. In *Proceedings of CAD’26*, pages 7–13, 2026. doi: 10.14733/cadconfP.2026.7-13.
- [11] Christoph Hoffmann, Vadim Shapiro, and Vijay Srinivasan. Geometric interoperability via queries. *Computer-Aided Design*, 46:148–159, 2014. doi: 10.1016/j.cad.2013.08.027.
- [12] Lutz Kettner, Kurt Mehlhorn, Sylvain Pion, Stefan Schirra, and Chee Yap. Classroom examples of robustness problems in geometric computations. *Computational Geometry*, 40(1):61–78, 2008. doi: 10.1016/j.comgeo.2007.06.003.
- [13] Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 7552–7579, 2024. doi: 10.52202/079017-0242.
- [14] Byung Chul Kim, Duhwan Mun, Soonhung Han, and Michael J. Pratt. A method to exchange procedurally represented 2D CAD model data using ISO 10303 STEP. *Computer-Aided Design*, 43(12):1717–1728, 2011. doi: 10.1016/j.cad.2011.07.006.
- [15] Byung Chul Kim, Hanra Lee, Mutahar Safdar, Tahir Abbas Jauhar, and Soonhung Han. Exchange of parametric assembly models based on neutral assembly constraints. *Concurrent Engineering*, 2019. doi: 10.1177/1063293X19869047.

- [16] Junhwan Kim, Michael J. Pratt, Raj G. Iyer, and Ram D. Sriram. Standardized data exchange of CAD models with design intent. *Computer-Aided Design*, 40(7):760–777, 2008. doi: 10.1016/j.cad.2007.06.014.
- [17] Jiri Kripac. A mechanism for persistently naming topological entities in history-based parametric solid models. *Computer-Aided Design*, 29(2):113–122, 1997. doi: 10.1016/S0010-4485(96)00040-1.
- [18] Robert Lipman and James Filliben. Testing implementations of geometric dimensioning and tolerancing in CAD software. *Computer-Aided Design and Applications*, 17(6):1241–1265, 2020. doi: 10.14733/cadaps.2020.1241-1265.
- [19] Robert R. Lipman and James J. Filliben. Guide to the NIST PMI CAD models and CAD system PMI modeling capability verification testing results. Technical Report NIST AMS 100-10, National Institute of Standards and Technology, 2017.
- [20] Yu Liu, Jingzhe Ni, Yiming Chen, Junqi Huang, Ruofeng Tong, Min Tang, and Peng Du. CADIR: A cross-backend editable intermediate representation for agentic CAD generation, 2026. v1 submitted 2026-08-01.
- [21] Felipe Machado, Norberto Malpica, and Susana Borromeo. Parametric CAD modeling for open source scientific hardware: Comparing OpenSCAD and FreeCAD python scripts. *PLOS ONE*, 14(12):e0225795, 2019. doi: 10.1371/journal.pone.0225795.
- [22] Liane Makatura, Michael Foshey, Bohan Wang, Felix Hähnlein, Pingchuan Ma, Bolei Deng, Megan Tjandrasuwita, Andrew Spielberg, Crystal Elaine Owens, Peter Yichen Chen, Allan Zhao, Amy Zhu, Wil J. Norton, Edward Gu, Joshua Jacob, Yifei Li, Adriana Schulz, and Wojciech Matusik. How can large language models help humans in design and manufacturing?, 2023.
- [23] Dimitrios Mallis, Ahmet Serdar Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. CAD-Assistant: Tool-augmented VLLMs as generic CAD task solvers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. doi: 10.1109/ICCV51701.2025.00684.
- [24] David Marcheix and Guy Pierra. A survey of the persistent naming problem. In *Proceedings of the Seventh ACM Symposium on Solid Modeling and Applications*, pages 13–22, 2002. doi: 10.1145/566282.566288.
- [25] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1): 100–107, 1998.
- [26] Duhwan Mun, Soonhung Han, Junhwan Kim, and Youchon Oh. A set of standard modeling commands for the history-based parametric approach. *Computer-Aided Design*, 35(13): 1171–1179, 2003. doi: 10.1016/S0010-4485(03)00022-8.
- [27] Chandrakana Nandi, James R. Wilcox, Pavel Panchekha, Taylor Blau, Dan Grossman, and Zachary Tatlock. Functional programming for compiling and decompiling computer-aided design. *Proceedings of the ACM on Programming Languages*, 2(ICFP):1–31, 2018. doi: 10.1145/3236794.

- [28] Chandrakana Nandi, Max Willsey, Adam Anderson, James R. Wilcox, Eva Darulova, Dan Grossman, and Zachary Tatlock. Synthesizing structured CAD models with equality saturation and inverse transformations. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 31–44, 2020. doi: 10.1145/3385412.3386012.
- [29] Michael J. Pratt and Junhwan Kim. Experience in the exchange of procedural shape models using ISO 10303 (STEP). In *Proceedings of the ACM Symposium on Solid and Physical Modeling (SPM)*, pages 229–238, 2006. doi: 10.1145/1128888.1128920.
- [30] Vladislav Pyatov, Gleb Bobrovskikh, Saveliy Galochkin, Nikita Boldyrev, Oleg Voynov, Alexander Filippov, Gonzalo Ferrer, Peter Wonka, and Evgeny Burnaev. CADFS: A big CAD program dataset and framework for computer-aided design with large language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [31] Ari Rappoport. An architecture for universal CAD data exchange. In *Proceedings of the ACM Symposium on Solid Modeling and Applications*, 2003. doi: 10.1145/781644.781648.
- [32] Daniel Ritchie, Paul Guerrero, R. Kenny Jones, Niloy J. Mitra, Adriana Schulz, Karl D. D. Willis, and Jiajun Wu. Neurosymbolic models for computer graphics. *Computer Graphics Forum*, 42(2):545–568, 2023. doi: 10.1111/cgf.14775.
- [33] Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. CAD-Recode: Reverse engineering CAD code from point clouds. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9801–9811, 2025. doi: 10.1109/ICCV51701.2025.00914.
- [34] Mutahar Safdar, Tahir Abbas Jauhar, Youngki Kim, Hanra Lee, Chiho Noh, Hyunki Kim, Inhwan Lee, Imgyu Kim, Soonjo Kwon, and Soonhung Han. Feature-based translation of CAD models with macro-parametric approach: issues of feature mapping, persistent naming, and constraint translation. *Journal of Computational Design and Engineering*, 7(5):603–614, 2020. doi: 10.1093/jcde/qwaa043.
- [35] O. W. Salomons, F. J. A. M. van Houten, and H. J. J. Kals. Review of research in feature-based design. *Journal of Manufacturing Systems*, 12(2):113–132, 1993. doi: 10.1016/0278-6125(93)90012-I.
- [36] Duygu Sap and Vadim Shapiro. On verification of interoperability of CAD systems with a focus on invariant properties. *Computer-Aided Design*, 115:256–266, 2019. doi: 10.1016/j.cad.2019.05.004.
- [37] Duygu Sap and Daniel P. Szabo. Representation and system-agnostic automated CAD interoperability testing based on model interchangeability. *Journal of Computer Languages*, 85:101365, 2025. doi: 10.1016/j.col.2025.101365.
- [38] Ari Seff, Yaniv Ovadia, Wenda Zhou, and Ryan P. Adams. SketchGraphs: A large-scale dataset for modeling relational geometry in computer-aided design, 2020.
- [39] J. J. Shah. Assessment of features technology. *Computer-Aided Design*, 23(5):331–343, 1991. doi: 10.1016/0010-4485(91)90027-T.

- [40] David Shumway, Jonathan Sadler, and John Salmon. Hybrid state transactional database for product lifecycle management features in a multi-engineer synchronous heterogeneous CAD environment. *Computer-Aided Design and Applications*, 15(1):34–46, 2018. doi: 10.1080/16864360.2017.1353736.
- [41] Robert B. Tilove. Set membership classification: A unified approach to geometric intersection problems. *IEEE Transactions on Computers*, C-29(10):874–883, 1980. doi: 10.1109/TC.1980.1675470.
- [42] Yan Wang and Bartholomew O. Nnaji. Geometry-based semantic ID for persistent and interoperable reference in feature-based parametric modeling. *Computer-Aided Design*, 37(10):1081–1093, 2005. doi: 10.1016/j.cad.2004.11.009.
- [43] Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic CAD construction from human design sequences. *ACM Transactions on Graphics*, 40(4), 2021. doi: 10.1145/3450626.3459818. SIGGRAPH 2021; preprint arXiv:2010.02392.
- [44] Rundi Wu, Chang Xiao, and Changxi Zheng. DeepCAD: A deep generative network for computer-aided design models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6752–6762, 2021. doi: 10.1109/ICCV48922.2021.00670.
- [45] Xiang Xu, Karl D. D. Willis, Joseph G. Lambourne, Chin-Yi Cheng, Pradeep Kumar Jayaraman, and Yasutaka Furukawa. SkexGen: Autoregressive generation of CAD construction sequences with disentangled codebooks. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *PMLR*, pages 24698–24724, 2022.
- [46] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 283–294, 2011. doi: 10.1145/1993498.1993532.
- [47] Qingnan Zhou, Eitan Grinspun, Denis Zorin, and Alec Jacobson. Mesh arrangements for solid geometry. *ACM Transactions on Graphics*, 35(4), 2016. doi: 10.1145/2897824.2925901.